

OpenCL 加速技术总结

目录

1.	术语	2
2.	简介	2
3.	软件框架	2
3.1.	CPU 算法移植为 OpenCL 算法的方法.....	3
3.1.1.	编写内核函数.....	4
3.1.2.	编写创建 OpenCL Buf 的函数.....	5
3.1.3.	编写执行 OpenCL 内核函数.....	6
3.1.4.	发送函数示例.....	7
3.2.	CPU 与 GPU 协同并行计算.....	8
4.	性能测试	9
4.1.	耗时测试	9
4.1.1.	高通 SDM660 平台.....	9
4.1.2.	高通 SDM665 平台.....	12
4.2.	功耗、资源使用情况	13
4.2.1.	高通 SDM665 平台.....	13
5.	OpenCL 开发调试.....	14
5.1.	查看 GPU 占用率	14
5.2.	打印内核函数中变量的值	14
5.3.	SnapdragonProfiler 调试工具.....	14
5.4.	设置合适的工作组大小	16
5.5.	OpenCL Buf 读写性能优化.....	17
5.6.	避免使用 long int 数据进行除法运算	17
5.7.	不适合 OpenCL 计算的情形	18

1. 术语

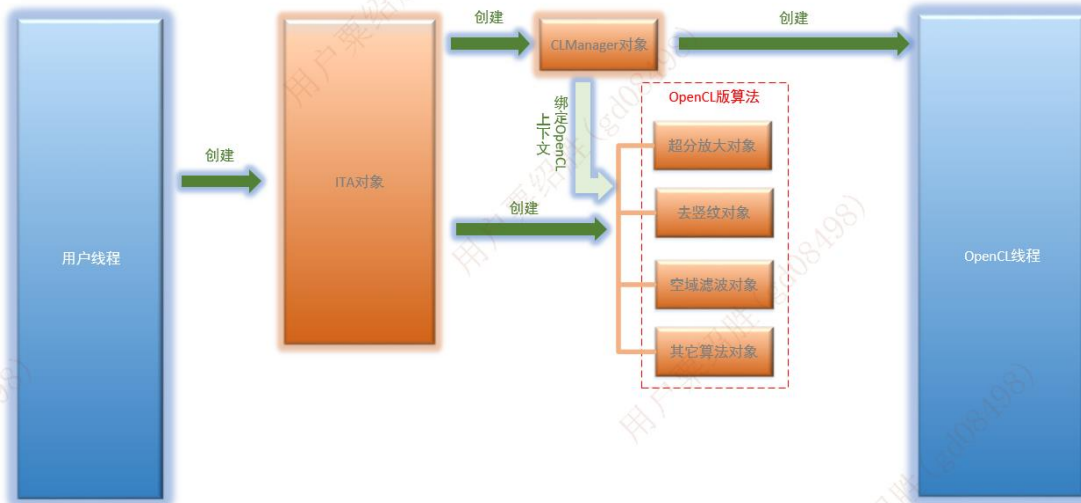
名称	说明
GPU	图形处理器 (Graphics Processing Unit)
OpenGL	开放图形库 (Open Graphics Library), 用于渲染 2D、3D 矢量图形的跨语言、跨平台的应用程序编程接口
OpenCL	开放运算语言 (Open Computing Language), 是第一个面向异构系统通用目的并行编程的开放式、免费标准
egl	egl 是 OpenGL ES 渲染 API 和本地窗口系统(native platform window system)之间的一个中间接口层

2. 简介

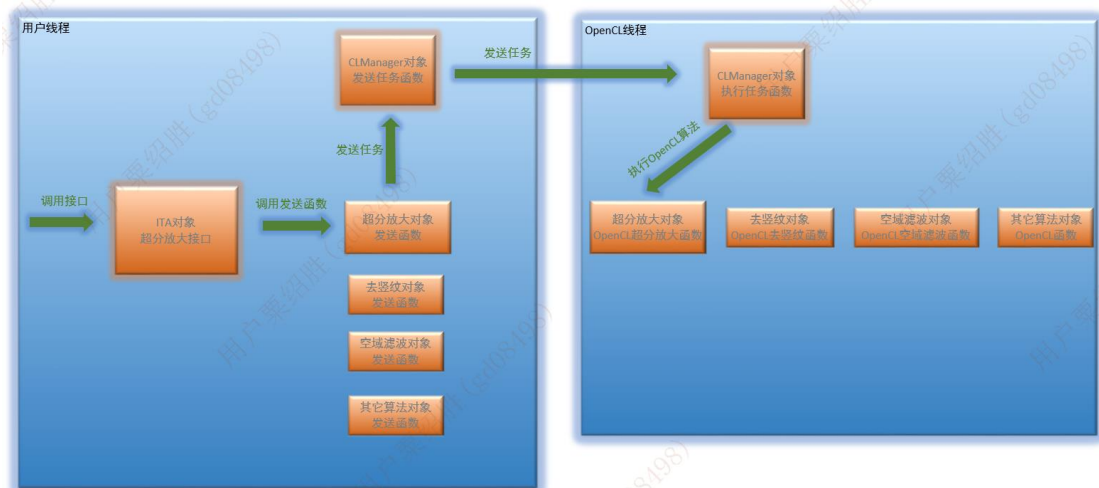
在 ZP29A 项目实现了去竖纹、滤波、调光、超分放大等算法的 OpenCL 加速，取得了一些成果，积累了一些 OpenCL 加速技术的经验。

3. 软件框架

OpenCL 使用命令队列的方式执行任务，同一时刻只能执行一个任务，并且所有 OpenCL API 必须在同一个线程中调用；因此设计了一个软件框架，所有与 OpenCL API 相关的任务由一个 OpenCL 线程负责执行。



ITA 模块初始化示意图



ITA 算法执行过程示意图

CLManager 对象: 负责启动一个 OpenCL 线程、创建 OpenCL 上下文，将各 OpenCL 版的算法与 OpenCL 上下文绑定；负责将用户线程中调用的算法发送到 OpenCL 线程中执行。

OpenCL 线程: 负责执行 CLManager 从用户线程发送过来的任务。

OpenCL 版算法: 各个算法对象，内部调用了 OpenCL API, 是 OpenCL 实现的算法；每个算法需实现一对函数——发送函数和 OpenCL 算法函数；发送函数运行于用户线程，OpenCL 算法函数运行于 OpenCL 线程。

3.1. CPU 算法移植为 OpenCL 算法的方法

OpenCL 代码可以抽象出几个通用接口：

- 1、initCLProgram(), 编译内核函数；
- 2、unInitCLProgram(), 释放程序资源函数；
- 3、createCLBuf(CreateCLBufArgs args), 创建 OpenCL Buf 函数；
- 4、releaseCLBuf(), 释放 OpenCL Buf 资源函数；
- 5、xxxCL(), 执行内核函数。

以空域滤波中的高斯滤波函数为例，介绍如何将 CPU 的算法移植为 OpenCL 的算法。

高斯滤波函数（CPU）：

```
void SpaceFilter::GaussianFilter(short *pus_dst, short *pus_src, int n_width, int n_height,
int n_win_wid, int* pos_weight_table, short *pus_src_pad)
{
    // 垫衬扩展范围
    m_assistanceCommon->PadMatrix(pus_src_pad, pus_src, n_width, n_height, n_win_wid,
n_win_wid);

    // 计算邻域内灰度加权和
```

```

short *CurRowPtr, *LastRowPtr, *NextRowPtr;
short *DstPixelPtr;
for (i = 0; i < n_height; i++)
{
    LastRowPtr = pus_src_pad + i * n_width_new + 1;
    CurRowPtr = LastRowPtr + n_width_new;
    for (j = 0; j < n_width; j++)
    {
        // for 循环语句块
        n_val_sum = (*(LastRowPtr - 1) * pos_weight_table[0] + *(LastRowPtr)*
pos_weight_table[1]
        + *(LastRowPtr + 1) * pos_weight_table[2] + *(CurRowPtr - 1) *
pos_weight_table[3]
        + *(CurRowPtr)* pos_weight_table[4] + *(CurRowPtr + 1) *
pos_weight_table[5]
        + *(NextRowPtr - 1) * pos_weight_table[6] + *(NextRowPtr)*
pos_weight_table[7]
        + *(NextRowPtr + 1) * pos_weight_table[8]);
        *DstPixelPtr = (short) (n_val_sum >> 12);
        LastRowPtr++;
        CurRowPtr++;
    }
}
}

```

3.1.1. 编写内核函数

首先找出函数中耗时大、可以并行计算的语句块，高斯滤波函数中的两个 for 循环符合 OpenCL 并行计算的条件。

将最里面一级的 for 循环语句块抽出来，作为内核函数的语句块，语句块用到的外部传入的 buf 指针增加到内核函数的形参中，外部传入的变量放到形参 args 中。

内核函数：

```

//args[0]: n_width; args[1]: n_width_new;\n",
__kernel void GaussianFilter(__global short* pus_dst,",
"
__global short* pus_src_pad, __global int*
pos_weight_table,",
"
__global int2* args)",
"{",
"    int i = get_global_id(1);",
"    int j = get_global_id(0);",
"    int2 local_args = *args;",

```

```

        "    int index = i * local_args.s0 + j;",
        "    short *LastRowPtr = pus_src_pad + i * local_args.s1 + 1 + j;",
        "    int8 local_pos_weight_table = vload8(0, pos_weight_table);",
        "    long n_val_sum = (*(LastRowPtr - 1) * local_pos_weight_table.s0 +
*(LastRowPtr) * local_pos_weight_table.s1",
        "        + *(LastRowPtr + 1) * local_pos_weight_table.s2 + *(CurRowPtr - 1) *
local_pos_weight_table.s3",
        "        + *(CurRowPtr) * local_pos_weight_table.s4 + *(CurRowPtr + 1) *
local_pos_weight_table.s5",
        "        + *(NextRowPtr - 1) * local_pos_weight_table.s6 + *(NextRowPtr) *
local_pos_weight_table.s7",
        "        + *(NextRowPtr + 1) * pos_weight_table[8]);",
        "    pus_dst[index] = (short)(n_val_sum >> 12);",
        "}",

```

3.1.2. 编写创建 OpenCL Buf 的函数

根据内核函数的参数，确定 buf 的读写权限、buf 的内存大小，创建 OpenCL Buf，并绑定到内核函数。

```

int SpaceFilterOpenCL::slotCreateCLBuf()
{
    int pus_dst_size = m_grayFilterCLBufArgs.pus_dst_size;
    int pus_src_pad_size = m_grayFilterCLBufArgs.pus_src_pad_size;
    int pos_weight_table_size = m_gaussianFilterCLBufArgs.pos_weight_table_size;
    // 创建 OpenCL Buf
    m_clPus_dst = clCreateBuffer(m_ctx.GPUContext, CL_MEM_WRITE_ONLY |
CL_MEM_HOST_READ_ONLY,
                                sizeof(short) * pus_dst_size, NULL, NULL);
    m_clPus_src_pad = clCreateBuffer(m_ctx.GPUContext, CL_MEM_READ_ONLY |
CL_MEM_HOST_WRITE_ONLY,
                                    sizeof(short) * pus_src_pad_size, NULL, NULL);
    m_clPos_weight_table = clCreateBuffer(m_ctx.GPUContext, CL_MEM_READ_ONLY |
CL_MEM_HOST_WRITE_ONLY,
                                          sizeof(int) * pos_weight_table_size, NULL,
NULL);
    m_clGaussianFilterArgs = clCreateBuffer(m_ctx.GPUContext, CL_MEM_READ_ONLY |
CL_MEM_HOST_WRITE_ONLY,
                                             sizeof(int) * GAUSSIANFILTER_ARGS_SIZE,
NULL, NULL);
    // 绑定到内核函数
    clSetKernelArg(m_kGaussianFilter, 0, sizeof(cl_mem), (void *) &m_clPus_dst);
    clSetKernelArg(m_kGaussianFilter, 1, sizeof(cl_mem), (void *) &m_clPus_src_pad);
    clSetKernelArg(m_kGaussianFilter, 2, sizeof(cl_mem), (void *) &m_clPos_weight_table);
    clSetKernelArg(m_kGaussianFilter, 3, sizeof(cl_mem), (void *) &m_clGaussianFilterArgs);
}

```

```
    return 0;
}
```

3.1.3. 编写执行 OpenCL 内核函数

该函数将数据从 CPU 更新到 GPU，将内核函数加入到命令队列中执行，等待内核函数执行完成后，将 GPU 计算结果更新到 CPU。

```
int SpaceFilterOpenCL::slotGaussianFilter_CL()
{
    short *pus_dst      = m_gaussianFilterArgs.pus_dst      ;
    int n_width         = m_gaussianFilterArgs.n_width     ;
    int n_height        = m_gaussianFilterArgs.n_height    ;
    short *pus_src_pad  = m_gaussianFilterArgs.pus_src_pad  ;
    int *pos_weight_table = m_gaussianFilterArgs.pos_weight_table;
    int pos_weight_table_size = m_gaussianFilterCLBufArgs.pos_weight_table_size;

    int n_len = n_width * n_height;

    int gaussianArgs[GAUSSIANFILTER_ARGS_SIZE] = { n_width, n_width_new, 0, 0};

    int eventSize = 5;
    cl_event event[eventSize];

    for (int i = 0; i < eventSize; i++)
    {
        event[i] = clCreateUserEvent(m_ctx.GPUContext, NULL);
    }
    // 从 CPU 更新数据到 GPU
    short *pus_src_pad_clmap = (short *)clEnqueueMapBuffer(m_ctx.cqCommandQueue,
m_clPus_src_pad,
                                                                    CL_TRUE, CL_MAP_WRITE,
0, sizeof(short) * n_width_new * n_height_new, 0, NULL, NULL, NULL);
    int *args_clmap = (int *)clEnqueueMapBuffer(m_ctx.cqCommandQueue,
m_clGaussianFilterArgs,
                                                                    CL_TRUE, CL_MAP_WRITE, 0,
sizeof(int) * GAUSSIANFILTER_ARGS_SIZE, 0, NULL, NULL, NULL);
    int *pos_weight_table_clmap = (int *)clEnqueueMapBuffer(m_ctx.cqCommandQueue,
m_clPos_weight_table,
                                                                    CL_TRUE, CL_MAP_WRITE,
0, sizeof(int) * pos_weight_table_size, 0, NULL, NULL, NULL);

    memcpy(pus_src_pad_clmap, pus_src_pad, sizeof(short) * n_width_new * n_height_new);
    clEnqueueUnmapMemObject(m_ctx.cqCommandQueue, m_clPus_src_pad, pus_src_pad_clmap, 0,
```

```

NULL, &event[0]);

    memcpy(args_clmap, gaussianArgs, sizeof(int) * GAUSSIANFILTER_ARGS_SIZE);
    clEnqueueUnmapMemObject(m_ctx.cqCommandQueue, m_clGaussianFilterArgs, args_clmap, 0,
NULL, &event[1]);

    memcpy(pos_weight_table_clmap, pos_weight_table, sizeof(int) * pos_weight_table_size);
    clEnqueueUnmapMemObject(m_ctx.cqCommandQueue, m_clPos_weight_table,
pos_weight_table_clmap, 0, NULL, &event[2]);

    clWaitForEvents(3, &event[0]);

    size_t WorkSize[2] = {(size_t)n_width, (size_t)n_height};
    size_t localWorkSize[2] = {(size_t)32, (size_t)32};
    // 执行内核函数
    clEnqueueNDRangeKernel(m_ctx.cqCommandQueue, m_kGaussianFilter, 2, NULL, WorkSize,
localWorkSize, 0, NULL, &event[3]);
    clWaitForEvents(1, &event[3]);

    // 将 GPU 计算结果更新到 CPU
    short *pus_dst_clmap = (short *)clEnqueueMapBuffer(m_ctx.cqCommandQueue, m_clPus_dst,
CL_TRUE, CL_MAP_READ, 0,
sizeof(short) * n_len, 0, NULL, NULL, NULL);
    memcpy(pus_dst, pus_dst_clmap, sizeof(short) * n_len);
    clEnqueueUnmapMemObject(m_ctx.cqCommandQueue, m_clPus_dst, pus_dst_clmap, 0, NULL,
&event[4]);
    clWaitForEvents(1, &event[4]);

    for(int i = 0; i < eventSize; i++) {
        clReleaseEvent(event[i]);
    }

    return 0;
}

```

至此，算法移植工作基本完成，剩下的工作就是为每个调用到 OpenCL API 的函数编写一个发送函数。

3.1.4. 发送函数示例

以高斯滤波算法为例，发送函数如下：

```

int SpaceFilterOpenCL::GaussianFilter_CL(GaussianFilterArgs args, CLFlag_E flag)
{
    int ret = -1;

```

```

if (m_ctx.pCLManager) {
    m_gaussianFilterArgs = args;

    CLManager *pManager = (CLManager *) m_ctx.pCLManager;
    //将 Y16 超分放大任务发送到 OpenCL 线程
    pManager->sendDoCLSignal(CLSIG_SPACEFILTER_GAUSSIANFILTER);

    if (flag == CLFLAG_BLOCK) {
        //等待任务执行完成
        ret = pManager->waitCL(CLSIG_SPACEFILTER_GAUSSIANFILTER);
    } else
    {
        ret = 0;
    }
}

return ret;
}

```

3.2. CPU 与 GPU 协同并行计算

该软件框架支持 CPU 与 GPU 协同并行计算，用户线程以非阻塞方式向 OpenCL 线程提交一个 OpenCL 任务，在 OpenCL 线程执行内核函数的同时，用户线程可以继续处理其它任务，实现 CPU 与 GPU 协同并行计算。

```

int clWorkSize = gImageLen * 1 / 2;
// 非阻塞方式，提交一个 OpenCL 任务后立即返回
g_mappingCLObject->Y16MappingAD2_CL(y8_buf, clWorkSize,
                                     y16_buf, Y16_BUF_LEN,
                                     pLinearMap, LINEARMAP_LEN,
                                     Y16_OFFSET, nBeta,
                                     nHistRange, HIST_OFFSET,
                                     CLFLAG_NOBLOCK);

// GPU 与 CPU 同时计算
// GPU 从 offset=0 计算前半数据，CPU 从 offset= clWorkSize 计算后半数据
for (i = clWorkSize; i < gImageLen; i += 4) {
    lValTmp = pLinearMap[(y16_buf[i] + Y16_OFFSET) >> 1] + ((nBeta *
pHistMap[(y16_buf[i] + Y16_OFFSET) >> 1] >> 10) * nHistRange >> 8);
    if (lValTmp < 0) {
        lValTmp = 0;
    } else if (lValTmp > 255) {
        lValTmp = 255;
    }
}

```



```

        y8_buf[i] = lValTmp;
        //2
        lValTmp = pLinearMap[(y16_buf[i + 1] + Y16_OFFSET) >> 1] + ((nBeta *
pHistMap[(y16_buf[i + 1] + Y16_OFFSET) >> 1] >> 10) * nHistRange >> 8);
        if (lValTmp < 0) {
            lValTmp = 0;
        } else if (lValTmp > 255) {
            lValTmp = 255;
        }
        y8_buf[i + 1] = lValTmp;
        //3
        lValTmp = pLinearMap[(y16_buf[i + 2] + Y16_OFFSET) >> 1] + ((nBeta *
pHistMap[(y16_buf[i + 2] + Y16_OFFSET) >> 1] >> 10) * nHistRange >> 8);
        if (lValTmp < 0) {
            lValTmp = 0;
        } else if (lValTmp > 255) {
            lValTmp = 255;
        }
        y8_buf[i + 2] = lValTmp;
        //4
        lValTmp = pLinearMap[(y16_buf[i + 3] + Y16_OFFSET) >> 1] + ((nBeta *
pHistMap[(y16_buf[i + 3] + Y16_OFFSET) >> 1] >> 10) * nHistRange >> 8);
        if (lValTmp < 0) {
            lValTmp = 0;
        } else if (lValTmp > 255) {
            lValTmp = 255;
        }
        y8_buf[i + 3] = lValTmp;
    }
    // CPU 等待 GPU 执行完
    g_mappingCLObject->waitCL(CLSIG_MAPPING_AD2);

```

4. 性能测试

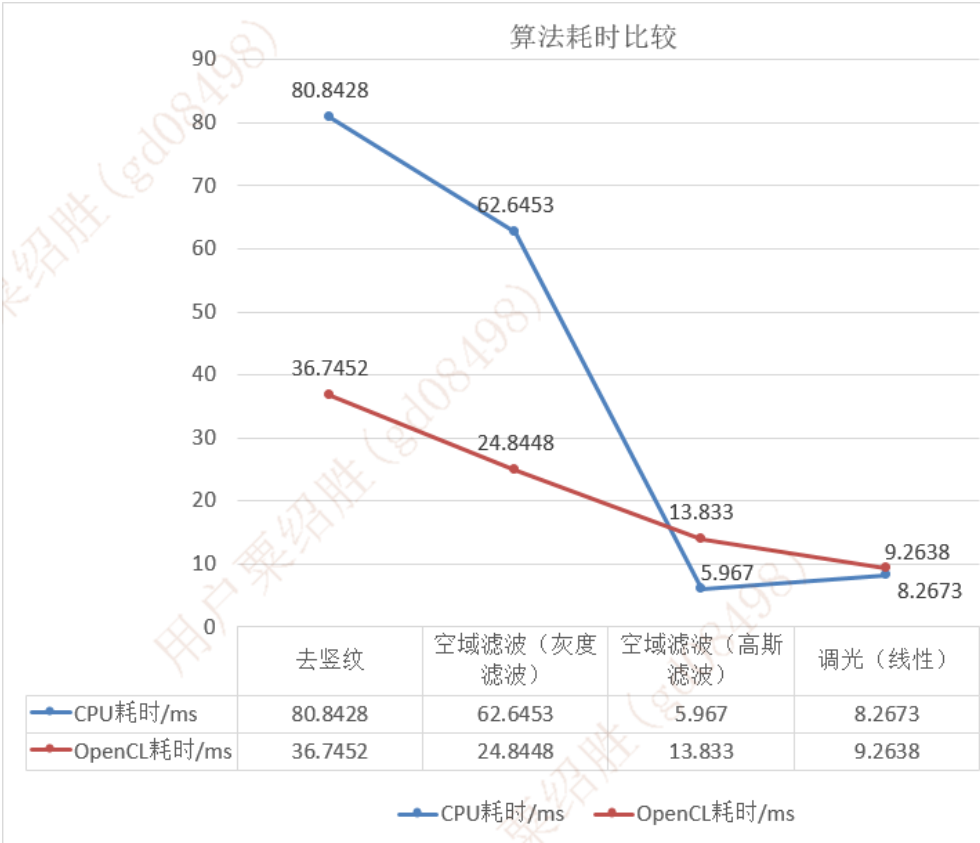
4.1. 耗时测试

4.1.1. 高通 SDM660 平台

ZC08A 项目使用了高通 SDM660 芯片平台，因此在高通 SDM660 平台上做了 OpenCL 加速性能测试。

1280*1024 分辨率下测试:

测试平台: 高通SDM660 芯片架构: ARMv8 测试图像分辨率: 1280*1024 测试日期: 2022/09/29 cpu参数: 4大核+4小核 GPU型号: adreno512 无项目环境下, 1000帧求平均计时, 单位: ms				
算法名称	函数名称	原始耗时	opengl加速	opengl加速比
两点校正	NUCbyTwoPoint	2.3631	/	
坏点替换	ReplaceBadPoint	0.0021	/	
时域滤波	TimeFF	13.7687	/	
去竖纹	RemoveSVN	80.8428	36.7452	2.20
去横纹	RemoveSHN	0.0009	/	
气体检测	HSM	0.0003	/	
空域滤波 (灰度滤波)	SpaceFilter	62.6453	24.8448	2.52
旋转	ImgRotation	0.0009	/	
镜像	ImgFlip	0.0003	/	
细节增强IIE	step1 GaussianFilter_16bit	/	/	
	step2 计算细节图像	/	/	
	step3 调光	/	/	
	step4 叠加细节	/	/	
	总计	/	/	
调光 (线性)	ModelDRT (线性)	8.2673	9.2638	0.89
Gamma校正	ImgGammaCorrect	/	/	
锐化	LaplaceSharpen	/	/	
Y8亮度对比度调节	ImgY8AdjustBC	/	/	
图像放大	ImgResize	/	/	
伪彩	PseudoColorMap	4.4019	/	
单帧耗时		175.8	100.6	1.75



超分放大算法因其循环体里没有条件分支、使用了浮点数计算，将其移植到 OpenCL 算法后加速效果较明显。

测试平台：高通SDM660 芯片架构：ARMv8 测试图像分辨率：1280*1024 测试日期：2022/11/15 cpu参数：4大核+4小核 GPU型号：adreno512 无项目环境下，500帧求平均计时，单位：ms			
函数名称	原始耗时	openc1加速	openc1加速比
bicubic (Y8)	720.7322	44.5574	16.18
bicubic (Y16)	649.6805	50.6476	12.83

400*300 分辨率下测试：

测试平台：高通SDM660 芯片架构：ARMv8 测试图像分辨率：400*300 测试日期：2022/11/15 cpu参数：4大核+4小核 GPU型号：adreno512 无项目环境下，1000帧求平均计时，单位：ms				
算法名称	函数名称	原始耗时	openc1加速	openc1加速比
超分放大 (Y16放大2倍)	bicubic (Y16)	59.2691	5.2728	11.24

在 ZC08A 真实运行环境下，由于使用了 egl，其占用了一部分 GPU 资源，OpenCL 加速效果不明显；

ZC08A 调光算法耗时测试：

测试环境： ZC08A 安卓设备 zc08 apk 程序（从探测器读取红外图像并进行处理，将图像显示到界面上）			
模块	耗时（ms）	运行前 GPU 占用率	运行时 GPU 负载
CPU(开启 egl)	17.5	0%	21%
CPU(关闭 egl)	16.5	0%	4%
OpenCL(开启 egl)	22.8	0%	50%
OpenCL(关闭 egl)	14.5	0%	15%

对于超分放大这种加速效果较明显的算法，在使用了 egl 的情况下，OpenCL 加速效果仍然较明显；

ZC08A 真实运行环境下（使用了 egl）测试：

测试平台：高通SDM660 芯片架构：ARMv8 测试图像分辨率：1280*1024 测试日期：2022/11/15 cpu参数：4大核+4小核 GPU型号：adreno512 ZC08A项目环境下运行egl，500帧求平均计时，单位：ms				
算法名称	函数名称	原始耗时	opengl加速	opengl加速比
超分放大（Y16放大2倍）	bicubic（Y16）	665.5246	71.062	9.37

4.1.2. 高通 SDM665 平台

ZC16A 项目使用了高通 SDM665 芯片平台，因此在高通 SDM665 平台上做了OpenCL 加速性能测试。

1280*1024 分辨率下测试：

测试平台：高通SDM665 芯片架构：ARMv8 测试图像分辨率：1280*1024 测试日期：2022/11/01 cpu参数：4大核+4小核 GPU型号：adreno610 无项目环境下，1000帧求平均计时，单位：ms				
算法名称	函数名称	原始耗时	opengl加速	opengl加速比
两点校正	NUCbyTwoPoint	2.198	/	
坏点替换	ReplaceBadPoint	0.002	/	
时域滤波	TimeFF	3.722	/	
去竖纹	RemoveSVN	77.8789	70.9919	1.10
去横纹	RemoveSHN	0	/	
气体检测	HSM	0	/	
空域滤波（灰度滤波）	SpaceFilter	61.1549	28.908	2.12
旋转	ImgRotation	0	/	
镜像	ImgFlip	0	/	
细节增强IIE	step1 GaussianFilter_16bit	/	/	
	step2 计算细节图像	/	/	
	step3 调光	/	/	
	step4 叠加细节	/	/	
	总计	/	/	
调光（线性）	ModelDRT（线性）	7.779	46.6069	0.17
Gamma校正	ImgGammaCorrect	/	/	
锐化	LaplaceSharpen	/	/	
Y8亮度对比度调节	ImgY8AdjustBC	/	/	
图像放大	ImgResize	/	/	
伪彩	PseudoColorMap	4.363	/	
单帧耗时		157.6	181.3	0.87

OpenCL运行时耗时打印：

NUC校正	1000帧平均耗时(ms)：7.856000
坏点替换	1000帧平均耗时(ms)：0.000000
时域滤波	1000帧平均耗时(ms)：7.856000
去竖纹	1000帧平均耗时(ms)：70.991997
去横纹	1000帧平均耗时(ms)：0.000000
气体检测	1000帧平均耗时(ms)：0.000000
空域滤波	1000帧平均耗时(ms)：28.908001
翻转	1000帧平均耗时(ms)：0.000000
旋转	1000帧平均耗时(ms)：0.000000
细节增强	1000帧平均耗时(ms)：0.000000
调光	1000帧平均耗时(ms)：46.606998
Gamma校正	1000帧平均耗时(ms)：0.000000
锐化	1000帧平均耗时(ms)：0.000000
Y8亮度对比度调节	1000帧平均耗时(ms)：0.000000
缩放	1000帧平均耗时(ms)：0.000000
伪彩	1000帧平均耗时(ms)：17.104000
单帧图像处理总耗时	1000帧平均耗时(ms)：181.356995

在高通 SDM665 平台上 OpenCL 加速效果不理想，并且会使到 CPU 的耗时增大。

4.2. 功耗、资源使用情况

4.2.1. 高通 SDM665 平台

运行 OpenCL 之后的功耗基本与原始（CPU 计算）功耗相同，GPU 占用率有提升，同时会降低 CPU 占用率，起到了分担 CPU 计算负荷的作用。

测试平台：高通665 芯片架构：ARMv8 测试图像分辨率：1280*1024 测试日期：2022/11/11 cpu参数：4大核+4小核 GPU型号：adreno610 无项目环境下，长时间运行ISP处理			
加速技术	功耗(单位：W)	CPU占用率（%）	GPU占用率（%）
原始	2.65	99	/
OpenCL	2.56	74	25

5. OpenCL 开发调试

5.1. 查看 GPU 占用率

```
cat /sys/class/kgsl/kgsl-3d0/gpu_busy_percentage
```

5.2. 打印内核函数中变量的值

OpenCL 调试手段有限，很多平台不支持 printf 打印，一旦内核函数中数据计算不正确时，很难定位问题。

一个有效的方法是调试时临时申请一段 OpenCL Buf, 用于存储内核函数中的变量值、临时计算结果，内核执行完成后，将其同步到 CPU 内存中，再在 CPU 上通过 printf 打印出来。

5.3. SnapdragonProfiler 调试工具

SnapdragonProfiler 调试工具由高通公司提供，可以调试高通平台的 CPU、GPU、DSP 等。

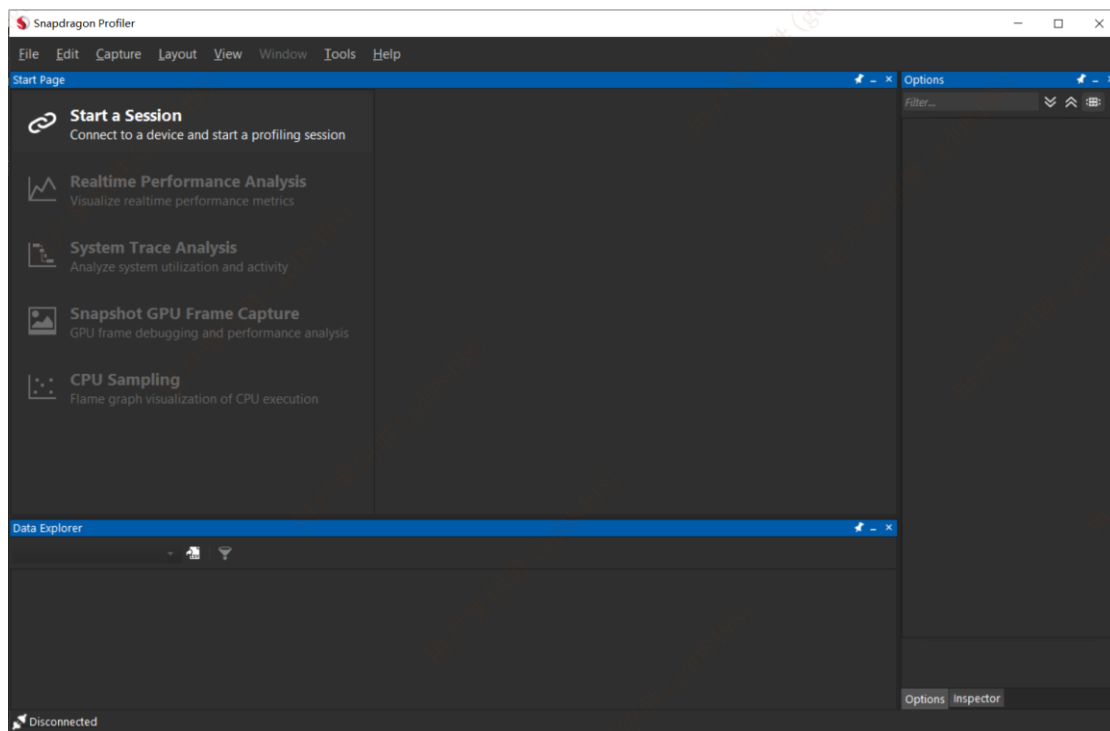
下载安装 gtk-sharp

<https://xamarin.azureedge.net/GTKforWindows/Windows/gtk-sharp-2.12.45.msi>

下载安装 snapdragon-profiler

<https://developer.qualcomm.com/software/snapdragon-profiler>

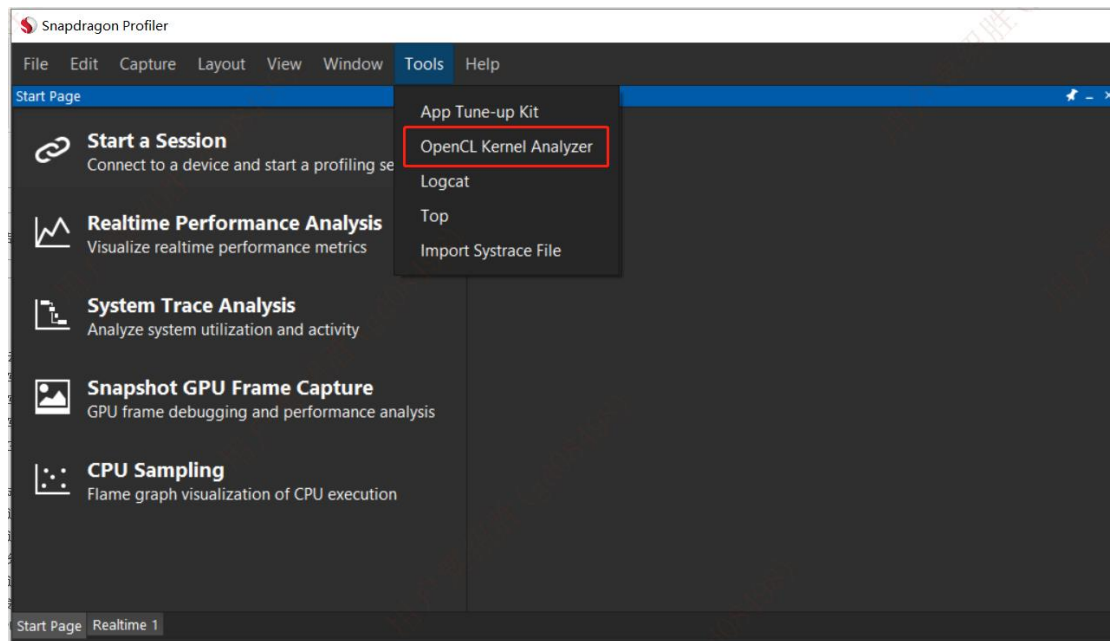
以管理员身份运行 snapdragon profiler



连上设备后，可以查看设备的 CPU、GPU、DSP 资源状态：



该工具支持 OpenCL 内核函数分析功能：



工具显示的内核函数资源使用情况：

```
- Instruction stats
- Main Body stats
- All Instructions: 262, 472 (rpt), ratio 1.80
- ALUs : 147, 159 (rpt), ratio 1.08
- Total NOPs : 90, 233 (rpt), ratio 2.59
- NOPs : 38, 108 (rpt), ratio 2.84
- Post-NOPs : 52, 125 (rpt), ratio 2.40
- EFUs : 1, 1 (rpt), ratio 1.00
- MOV class : 53, 56 (rpt), ratio 1.06
- MOVs : 53, 56 (rpt), ratio 1.06
- CMP : 11, 14 (rpt), ratio 1.27
- SEL : 9, 12 (rpt), ratio 1.33
- Loads/Stores : 20, 26 (rpt), ratio 1.30
- ldg : 18, 24 (rpt), ratio 1.33
- stg : 2, 2 (rpt), ratio 1.00
- ldgsize : 74
- stgsize : 4
- memory_ldAllsize : 74
- memory_stAllsize : 4
- EI Position : -1
- Flow Instrs : 3
- Short sync flags: 2
- Long sync flags : 10
- SY before ei pos: 10
- Full Registers : 7
- Half Registers : 1
- uGPR Registers : 4
- Unified Regs : 8
- Total footprint : 128
- Wave sizes : 32
- Max num of waves: 12
- Maximal Waves : 12 (A6x)
```

5.4. 设置合适的工作组大小

显示设置工作组大小，可以获得性能提升。


```
size_t WorkSize[2] = {(size_t)n_width, (size_t)n_height};
size_t localWorkSize[2] = {(size_t)32, (size_t)32};
clEnqueueNDRangeKernel(m_ctx.cqCommandQueue, m_kGrayFilter, 2, NULL, WorkSize, localWorkSize, 0, NULL, &event[4]);
```

工作组的大小要设置为多少，可以参考 CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE 的值，在代码中可以通过 API 查询：

```
clGetKernelWorkGroupInfo(m_kGrayFilter,
    m_ctx.cdDevice,
    CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
    sizeof(size_t),
    &preferredWorkGroupSize,
    NULL );
printf("OpenCL kernel GrayFilter CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE: %d\n", preferredWorkGroupSize);
```

工作组大小取其整数倍数的大小，然后通过修改 X 方向、Y 方向的大小，不断测试效果，确定合适的工作组大小。

5.5. OpenCL Buf 读写性能优化

读写内存时使用 clEnqueueMapBuffer 接口，将 GPU 内存映射到 cpu 指针上进行更新，与 clEnqueueWriteBuffer 接口相比，可以有效降低 GPU 占用率。

5.6. 避免使用 long int 数据进行除法运算

在空域滤波算法中，n_mat_value_sum、n_mat_weight_sum 变量设置为 long int 和 int 类型时的耗时差别明显；

使用 long int 类型的耗时：

space filter proc time (ms): 155.914993

改为 int 类型的耗时：

space filter proc time (ms): 24.603001

```

/* for (m = 0; m <= 2; m++)",
/* {",
/*     for (n = 0; n <= 2; n++)",
/*     {",
/*         n_gray_diff = (int) abs(pus_src_pad[temp + local_args.s3] - pus_src_pad[temp + m * local_args.s1 + n]);",
/*         if (n_gray_diff >= local_args.s2)",
/*         {",
/*             n_gray_diff = local_args.s2 - 1;",
/*         }",
/*         n_weight = gray_weight_table[n_gray_diff];",
/*         mat_weight_sum += n_weight;",
/*         mat_value_sum += (pus_src_pad[temp + m * local_args.s1 + n] * n_weight);",
/*     }",
/* }",
/* if (mat_weight_sum > 0) ",
/* {",
/*     pus_dst[index] = (short) mat_value_sum / mat_weight_sum;",
/* }",

```

5.7. 不适合 OpenCL 计算的情形

循环体中依赖上一次的计算结果的情形不适合 OpenCL 计算；

```

for (i = 0; i < n_len; i++)
{
    nPixVal = (int) (pusSrcImg[i] + g_nOffset);
    nGraySum += nPixVal;

    if (nPixVal < n0rimin)
    {
        n0rimin = nPixVal;
    } else if (nPixVal > n0rimax)
    {
        n0rimax = nPixVal;
    }
}

```

移植到 OpenCL 后出现竞态时不适合 OpenCL 计算；

```

for (i = 0; i < n_len; i++)
{
    nPixVal = (int)(pusSrcImg[i] + g_nOffset);
    pHist[nPixVal]++;
    nGraySum += nPixVal;

    if (nPixVal < nOrimin)
    {
        nOrimin = nPixVal;
    }
    else if (nPixVal > nOrimax)
    {
        nOrimax = nPixVal;
    }
}

```

移植到 OpenCL 后会产生竞态，需加原子操作锁，降低性能：

```

__kernel __attribute__((work_group_size_hint(128, 1, 1))) ",
void getHist(__global unsigned int* pHist, __global short* pusSrcImg, ",
            __global int* args)",
"{",
    int i = get_global_id(0);",
    int nPixVal = (int)(pusSrcImg[i] + args[0]);",
    atomic_fetch_add((atomic_uint *) (pHist + nPixVal), 1);",
"}",

```